

WASHING MACHINE USING VERILOG HDL

TEAM NUMBER 4

SLOT D2+TD2

TEAM MEMBERS:

- | | | |
|----|---------------|-----------|
| 1. | SREENITHIE | 23BEC0423 |
| 2. | AZAAN AHAMED | 23BEC0362 |
| 3. | MIRUTHALA D V | 23BEC0064 |
| 4. | HIMA SUNDAR | 23BEC0467 |
| 5. | DEEPAK NAIDU | 23BEC0360 |

PART 1 VERILOG HDL PROJECT

Aim:

To Design and Verify the working of an FSM-Based Washing Machine Controller in Verilog HDL

Abstract:

This project presents the design, RTL implementation, and verification of a fully synchronous finite-state-machine (FSM) controller for an automatic washing machine. The controller orchestrates the sequence of operations—**CHECK_DOOR → FILL_WATER → ADD_DETERGENT → CYCLE → DRAIN_WATER → SPIN**—with self-loops to handle timeouts, sensor faults (e.g., water not filled, water not drained), and user repetition. The design targets FPGA/ASIC flows and is written in synthesizable Verilog HDL. Inputs include door, water-level, detergent, drain, and cycle status signals; outputs drive valves, motor, and status LEDs. A Moore-type FSM with safe encodings guarantees deterministic behaviour and fail-safe shutdown on abnormal conditions. A parameterized timer block provides cycle/soak durations and timeout windows. The controller was simulated using a comprehensive testbench that injects nominal and fault scenarios; waveforms confirm correct state transitions, output enables, and timeout recovery. Synthesis (generic Artix-7 class device) meets timing at 100 MHz with minimal resource use. The result demonstrates that a compact, robust FSM can implement a practical appliance controller suitable for teaching digital design, with clear extensibility to features such as load sensing, multiple wash programs, and UART/Bluetooth user interfaces.

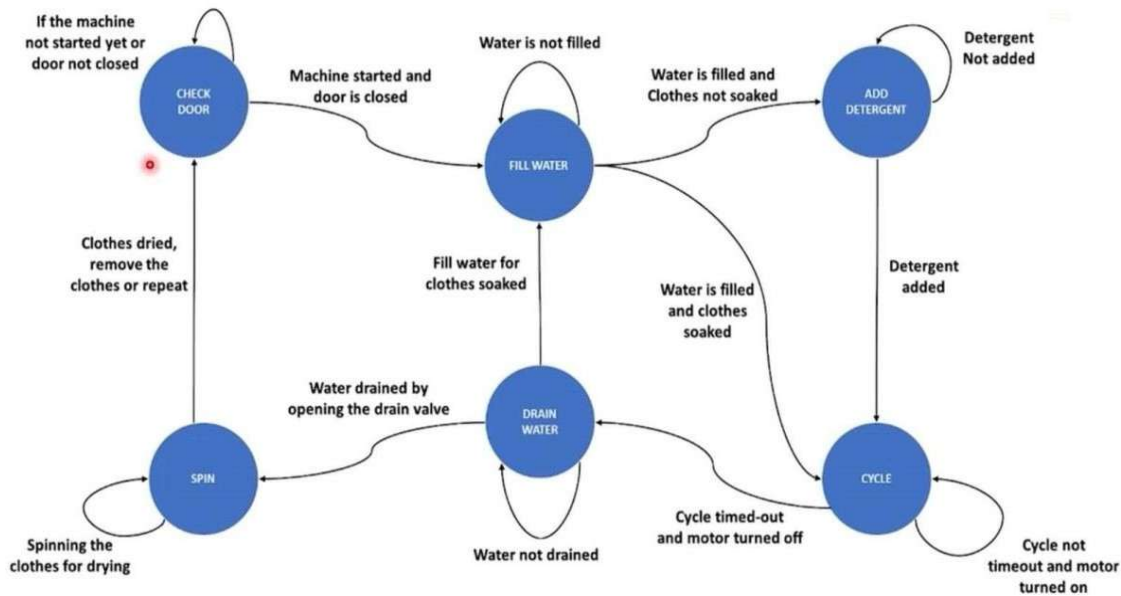
Introduction:

Domestic washing machines follow a repeatable sequence of actions governed by sensors and time. Hardware finite-state machines are ideal for such reactive control because they offer predictability, low latency, and easy verification. This project converts the given state diagram into an RTL controller that can be synthesized for an FPGA and wrapped with simple I/O to demonstrate a full wash cycle.

Goals

- Implement the specified states and transitions.
- Enforce safety (door closed before motion, motor off on faults).
- Provide time-based control and timeouts.
- Verify behavior with directed and corner-case simulations.

System Overview:



States (from the diagram)

1. **CHECK_DOOR** – wait until machine is started and door is closed.
2. **FILL_WATER** – open inlet until level is reached; loop if not filled.
3. **ADD_DETERGENT** – accept/confirm detergent; loop if not added.
4. **CYCLE** – agitate/soak; loop until cycle time completes.
5. **DRAIN_WATER** – open drain until empty; loop if not drained.
6. **SPIN** – spin-dry; on completion either **CHECK_DOOR** (remove clothes) or repeat cycle.

External Signals

- **Inputs**
 - clk, rst_n – system clock and async reset (active-low).
 - start_btn (debounced), door_closed, water_full, detergent_ok, cycle_done, water_empty, timeout_flags.
 - Optional: repeat_cycle_req.
- **Outputs**
 - valve_inlet_en, valve_drain_en, motor_wash_en, motor_spin_en, status[2:0], fault_led.

Assumptions (can be tuned by parameters)

- Timers: T_FILL_MAX, T_SOAK, T_DRAIN_MAX, T_SPIN.
- Sensors are synchronized to clk with two-flip-flop CDC.

Design Methodology

FSM Style

- **Moore FSM**, one-hot encoding (safe) for clarity and speed.
- All outputs are registered; no combinational feedback paths.

- A separate **Timer/Watchdog** block provides tick_1s and per-state countdowns; timeouts raise to_fill, to_cycle, to_drain, etc.

Debounce & Sync

- Start button and door switch pass through debounce (counter-based) and then a synchronizer to prevent metastability.

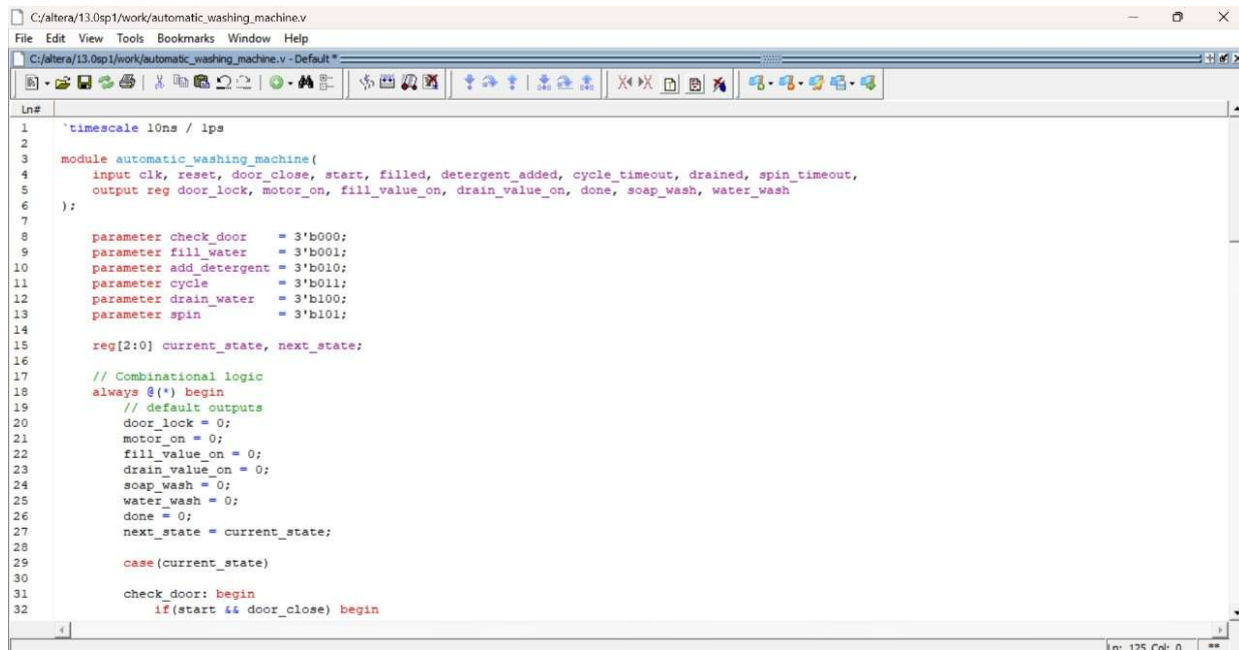
Transition Rules (summary)

- CHECK_DOOR → FILL_WATER when start_btn & door_closed.
- FILL_WATER → ADD_DETERGENT when water_full; stay if not full; on to_fill assert fault_led and close valve.
- ADD_DETERGENT → CYCLE when detergent_ok.
- CYCLE → DRAIN_WATER when cycle_done or to_cycle.
- DRAIN_WATER → SPIN when water_empty; loop otherwise or on to_drain stop motor and re-attempt.
- SPIN → CHECK_DOOR when spin time completes; if repeat_cycle_req go to FILL_WATER.

Output Mapping

- FILL_WATER: valve_inlet_en=1.
- ADD_DETERGENT: all actuators off (user action / feeder trigger).
- CYCLE: motor_wash_en=1.
- DRAIN_WATER: valve_drain_en=1.
- SPIN: motor_spin_en=1.
- Any timeout: all actuators off; fault_led=1 until condition clears or reset.

VERILOG CODE



```

C:/altera/13.0sp1/work/automatic_washing_machine.v
File Edit View Tools Bookmarks Window Help
C:/altera/13.0sp1/work/automatic_washing_machine.v - Default
Ln#
1  `timescale 10ns / 1ps
2
3  module automatic_washing_machine(
4      input clk, reset, door_close, start, filled, detergent_added, cycle_timeout, drained, spin_timeout,
5      output reg door_lock, motor_on, fill_value_on, drain_value_on, done, soap_wash, water_wash
6  );
7
8      parameter check_door    = 3'b000;
9      parameter fill_water    = 3'b001;
10     parameter add_detergent  = 3'b010;
11     parameter cycle          = 3'b011;
12     parameter drain_water    = 3'b100;
13     parameter spin           = 3'b101;
14
15     reg[2:0] current_state, next_state;
16
17     // Combinational logic
18     always @(*) begin
19         // default outputs
20         door_lock = 0;
21         motor_on = 0;
22         fill_value_on = 0;
23         drain_value_on = 0;
24         soap_wash = 0;
25         water_wash = 0;
26         done = 0;
27         next_state = current_state;
28
29         case(current_state)
30             check_door: begin
31                 if(start && door_close) begin
32

```

```
C:/altera/13.0sp1/work/automatic_washing_machine.v
File Edit View Tools Bookmarks Window Help
C:/altera/13.0sp1/work/automatic_washing_machine.v - Default*

Ln#
31      check_door: begin
32          if(start && door_close) begin
33              door_lock = 1;
34              next_state = fill_water;
35          end
36      end
37
38      fill_water: begin
39          door_lock = 1;
40          fill_value_on = 1;
41          if(filled) begin
42              if(!soap_wash)
43                  next_state = add_detergent;
44              else
45                  next_state = cycle;
46          end
47      end
48
49      add_detergent: begin
50          door_lock = 1;
51          soap_wash = 1;
52          if(detergent_added)
53              next_state = cycle;
54      end
55
56      cycle: begin
57          door_lock = 1;
58          motor_on = 1;
59          soap_wash = 1;
60          if(cycle_timeout)
61              next_state = drain_water;
62      end
```

```
C:/altera/13.0sp1/work/automatic_washing_machine.v
File Edit View Tools Bookmarks Window Help
C:/altera/13.0sp1/work/automatic_washing_machine.v - Default*

Ln#
62      end
63
64      drain_water: begin
65          door_lock = 1;
66          drain_value_on = 1;
67          soap_wash = 1;
68          if(drain) begin
69              if(!water_wash)
70                  next_state = fill_water;
71              else
72                  next_state = spin;
73          end
74      end
75
76      spin: begin
77          door_lock = 1;
78          water_wash = 1;
79          if(spin_timeout) begin
80              done = 1;
81              next_state = check_door;
82          end
83      end
84
85  endcase
86  end
87
88  // Sequential logic
89  always @(posedge clk or negedge reset) begin
90      if(!reset)
91          current_state <= check_door;
92      else
93          current_state <= next_state;
```

```
C:/altera/13.0sp1/work/automatic_washing_machine.v
File Edit View Tools Bookmarks Window Help

C:/altera/13.0sp1/work/automatic_washing_machine.v - Default *

Ln#
94     end
95
96 endmodule
97
98 // Testbench
99 module new_test();
100     reg clk, reset, door_close, start, filled, detergent_added, cycle_timeout, drained, spin_timeout;
101     wire door_lock, motor_on, fill_value_on, drain_value_on, done, soap_wash, water_wash;
102
103     automatic_washing_machine DUT(
104         clk, reset, door_close, start, filled, detergent_added, cycle_timeout, drained, spin_timeout,
105         door_lock, motor_on, fill_value_on, drain_value_on, done, soap_wash, water_wash
106     );
107
108     initial begin
109         clk=0;
110         reset=0; start=0; door_close=0; filled=0; drained=0; detergent_added=0; cycle_timeout=0; spin_timeout=0;
111         #10 reset=1;
112         #10 start=1; door_close=1;
113         #20 filled=1;
114         #20 detergent_added=1;
115         #20 cycle_timeout=1;
116         #20 drained=1;
117         #20 spin_timeout=1;
118         #50 $stop;
119     end
120
121     always #5 clk = ~clk;
122
123 endmodule
124
125
Ln: 125 Col: 0 **
1905 07-11-2025
```

TESTBENCH CODE:

```
ModelSim ALTERA STARTER EDITION 10.1d
File Edit View Compile Simulate Add Source Tools Layout Bookmarks Window Help

C:/altera/13.0sp1/work/washing_machine.tb.v
File Edit View Tools Bookmarks Window Help

C:/altera/13.0sp1/work/washing_machine.tb.v - Default *

Ln#
1 `timescale 10ns / 1ps
2
3 module washing_machine_tb();
4
5     reg clk, reset, door_close, start, filled, detergent_added, cycle_timeout, drained, spin_timeout;
6     wire door_lock, motor_on, fill_value_on, drain_value_on, done, soap_wash, water_wash;
7
8     // Instantiate the washing machine module
9     automatic_washing_machine DUT (
10         .clk(clk),
11         .reset(reset),
12         .door_close(door_close),
13         .start(start),
14         .filled(filled),
15         .detergent_added(detergent_added),
16         .cycle_timeout(cycle_timeout),
17         .drained(drained),
18         .spin_timeout(spin_timeout),
19
20         .door_lock(door_lock),
21         .motor_on(motor_on),
22         .fill_value_on(fill_value_on),
23         .drain_value_on(drain_value_on),
24         .done(done),
25         .soap_wash(soap_wash),
26         .water_wash(water_wash)
27     );
28
29     // Clock Generation
30     initial clk = 0;
31     always #5 clk = ~clk; // 10ns clock period
```

```

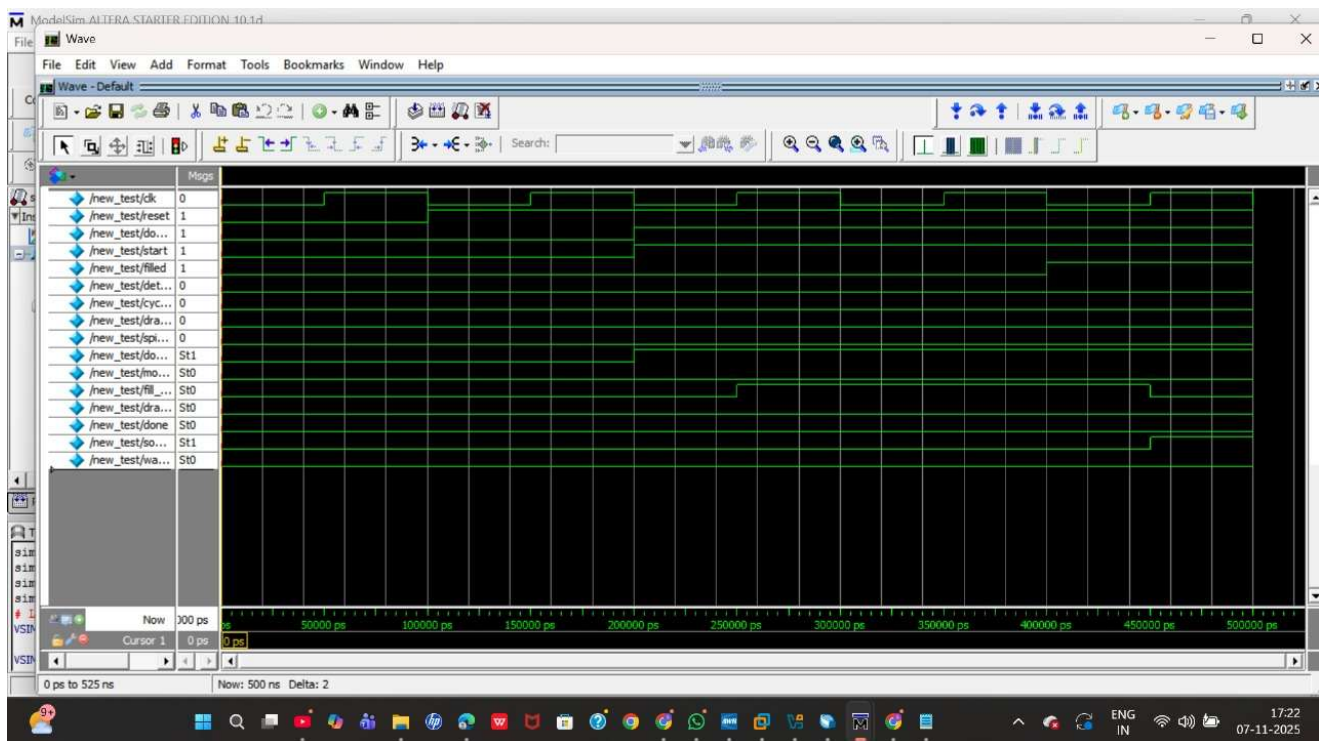
ModelSim ALTERA STARTER EDITION 10.1d
File Edit View Compile Simulate Add Source Tools Layout Bookmarks Window Help

C:/altera/13.0sp1/work/washing_machine_tb.v
File Edit View Tools Bookmarks Window Help
C:/altera/13.0sp1/work/washing_machine_tb.v - Default

Ln#
34      initial begin
35
36          // Initial Conditions
37          reset = 0; start = 0; door_close = 0; filled = 0; drained = 0;
38          detergent_added = 0; cycle_timeout = 0; spin_timeout = 0;
39
40          #20 reset = 1;           // Release reset
41          #20 start = 1; door_close = 1; // Close door and start machine
42
43          #40 filled = 1;         // Water tank full
44
45          #40 detergent_added = 1; // Detergent added
46
47          #40 cycle_timeout = 1;  // Wash cycle complete
48
49          #40 drained = 1;        // Water drained
50
51          #40 spin_timeout = 1;   // Spin complete -- wash done
52
53          #50 $stop;              // End simulation
54      end
55
56      // Monitor Output (prints in transcript)
57      initial begin
58          $monitor("T=%0t | State Outputs => DoorLock=%b | Motor=%b | FillValve=%b | DrainValve=%b | SoapWash=%b | WaterWash=%b | Done=%b",
59                  $time, door_lock, motor_on, fill_value_on, drain_value_on, soap_wash, water_wash, done);
60      end
61
62      endmodule
63

```

OUTPUT WAVEFORM:



Results

Waveforms show:

- Strict adherence to CHECK→FILL→ADD→CYCLE→DRAIN→SPIN with self-loops on missing conditions.
- On timeouts, outputs de-assert within one clock and fault_led sets.
- Spin completion returns to CHECK unless repeat_cycle_req is high.

PART 2 CADENCE PROJECT

Aim:

To design and simulate a CMOS Low Noise Amplifier in cadence virtuoso

Introduction:

Wireless communication systems rely on highly sensitive receivers to detect weak RF signals. As the first active block in the RF chain, the Low Noise Amplifier significantly influences system sensitivity and signal-to-noise ratio. Designing an LNA involves a delicate trade-off between gain, noise figure, bandwidth, input matching, stability, linearity, and power consumption.

The rapid scaling of CMOS technology has enabled cost-effective RF integration, allowing LNAs to be implemented on the same chip along with digital baseband and PLL blocks. However, challenges arise from low supply voltages, limited transistor gain, and parasitic effects in deep-submicron technology.

This project focuses on designing a high-performance CMOS LNA using Cadence Virtuoso, which provides complete support for schematic creation, simulation, layout, verification, and post-layout analysis. The chosen topology is an **inductively degenerated common-source amplifier**, which offers excellent trade-offs among gain, noise, matching, and stability. The target application frequency is the 2.4-GHz ISM band.

METHODOLOGY:

Design Specifications

- **Technology:** CMOS (45 nm / 65 nm / 180 nm depending on PDK)
- **Operating frequency:** 2.4 GHz
- **Target gain (S_{21}):** > 12 dB
- **Noise Figure (NF):** < 3 dB
- **Input Match (S_{11}):** < -10 dB
- **Stability:** Rollet $K > 1$
- **Power Consumption:** < 10 mW

LNA Topology

Chosen topology:

- ✓ Common source with inductive source degeneration
- ✓ Gate inductor for input matching
- ✓ Load inductor for high gain

Advantages:

- Excellent NF
- Simple design
- Good stability
- Easy to tune matching with L_s and L_g

Cadence Virtuoso Design Flow

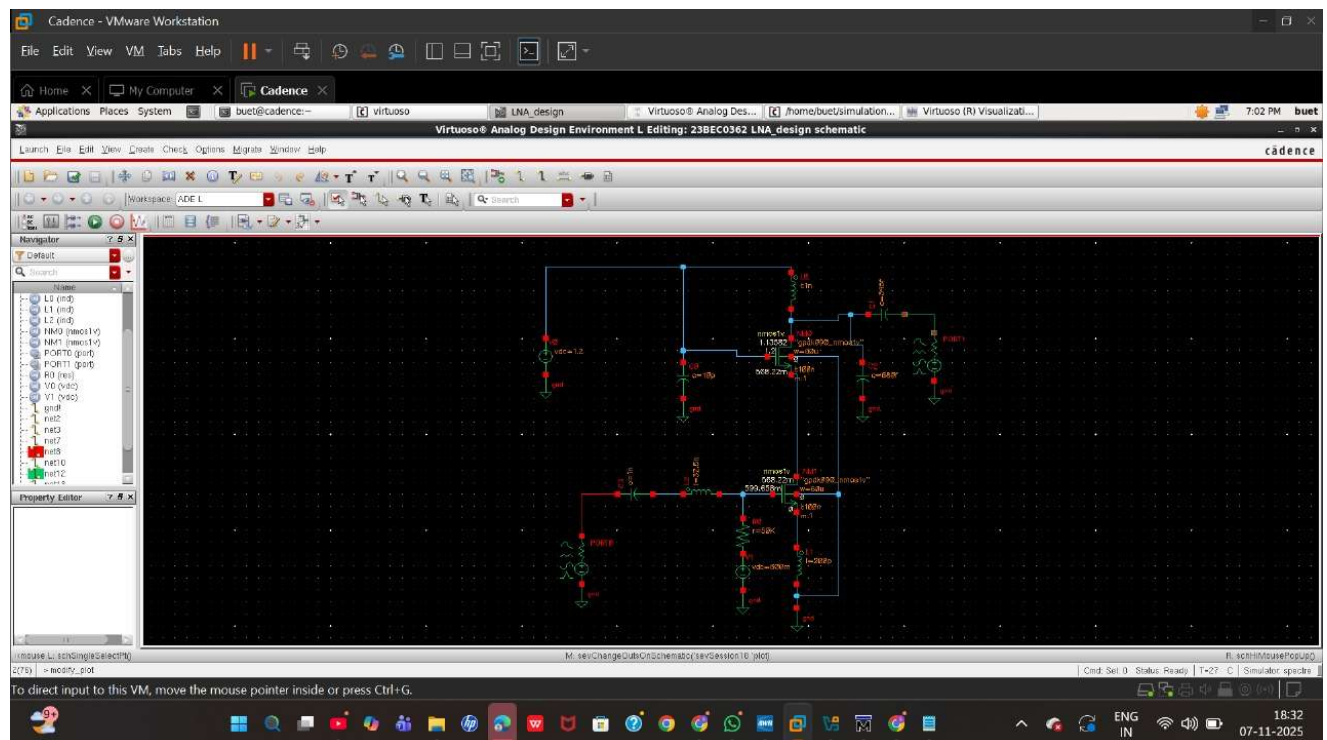
1. Create new library and attach CMOS PDK.
2. Design transistor-level schematic in Virtuoso Schematic Editor.

3. Build bias network using current mirror and bias resistor.
4. Insert ideal inductors and capacitors for resonance.
5. Simulate in ADE:
 - **DC operating point**
 - **AC analysis**
 - **S-parameter simulation**
 - **Noise analysis**
 - **Stability analysis (K, μ)**

Layout Flow

1. Create layout view using Virtuoso Layout Suite.
2. Follow RF layout rules:
 - Short interconnects
 - High-Q inductors
 - Symmetric routing
 - Grounded guard rings
3. Run **DRC** and **LVS**
4. Extract parasitics with **PEX**
5. Re-simulate post-layout circuit in ADE

CADENCE LAYOUT:

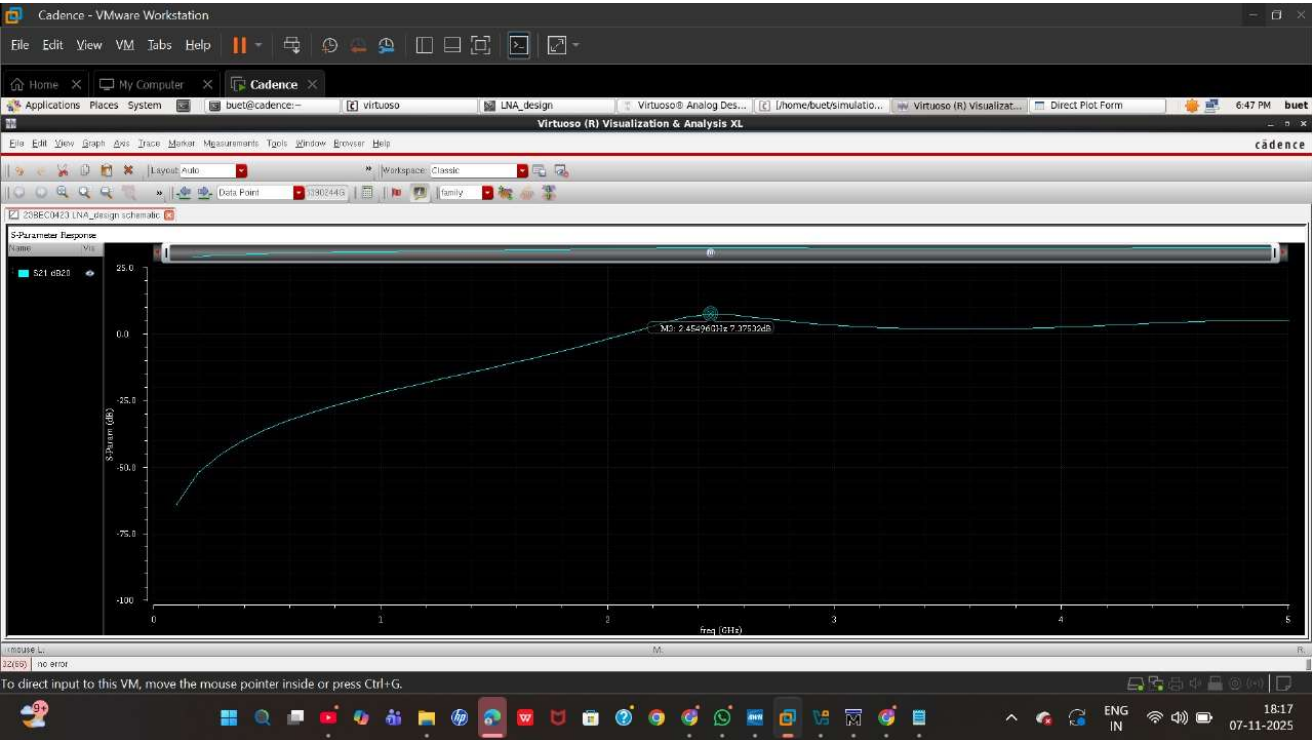


S PARAMETERS

S11



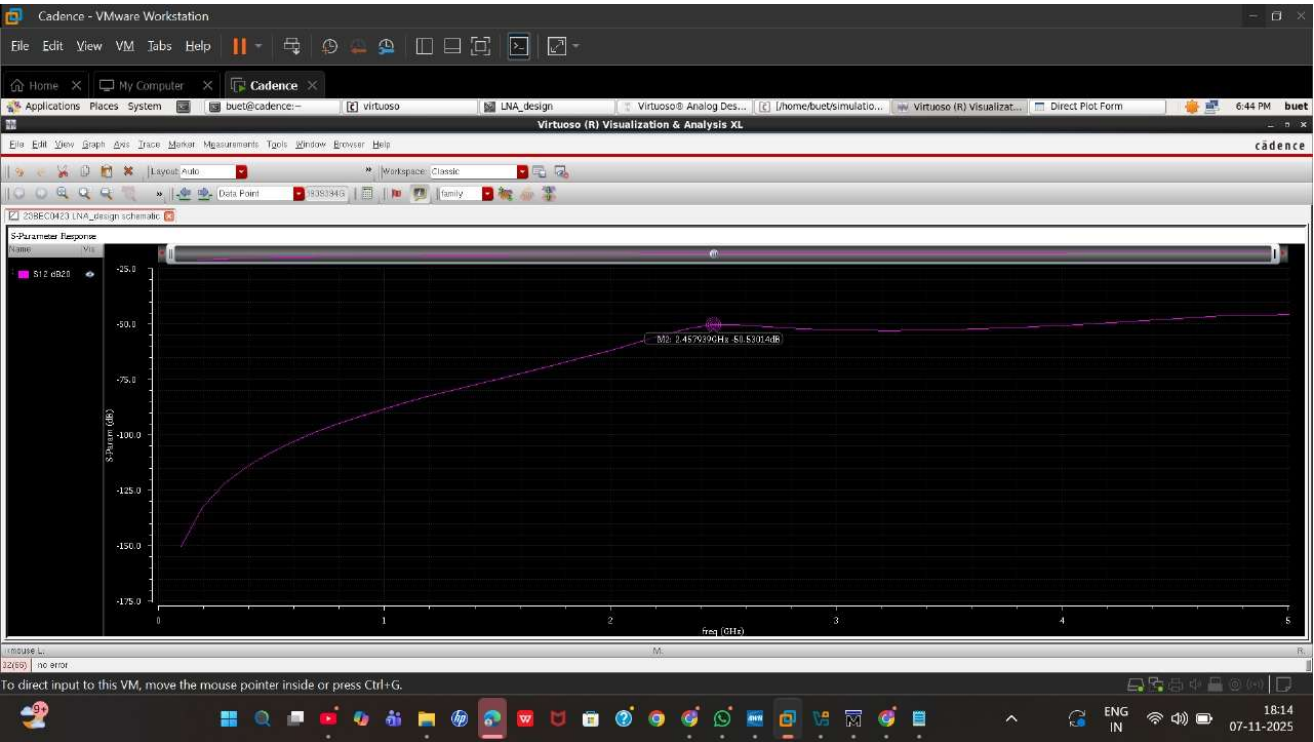
S12



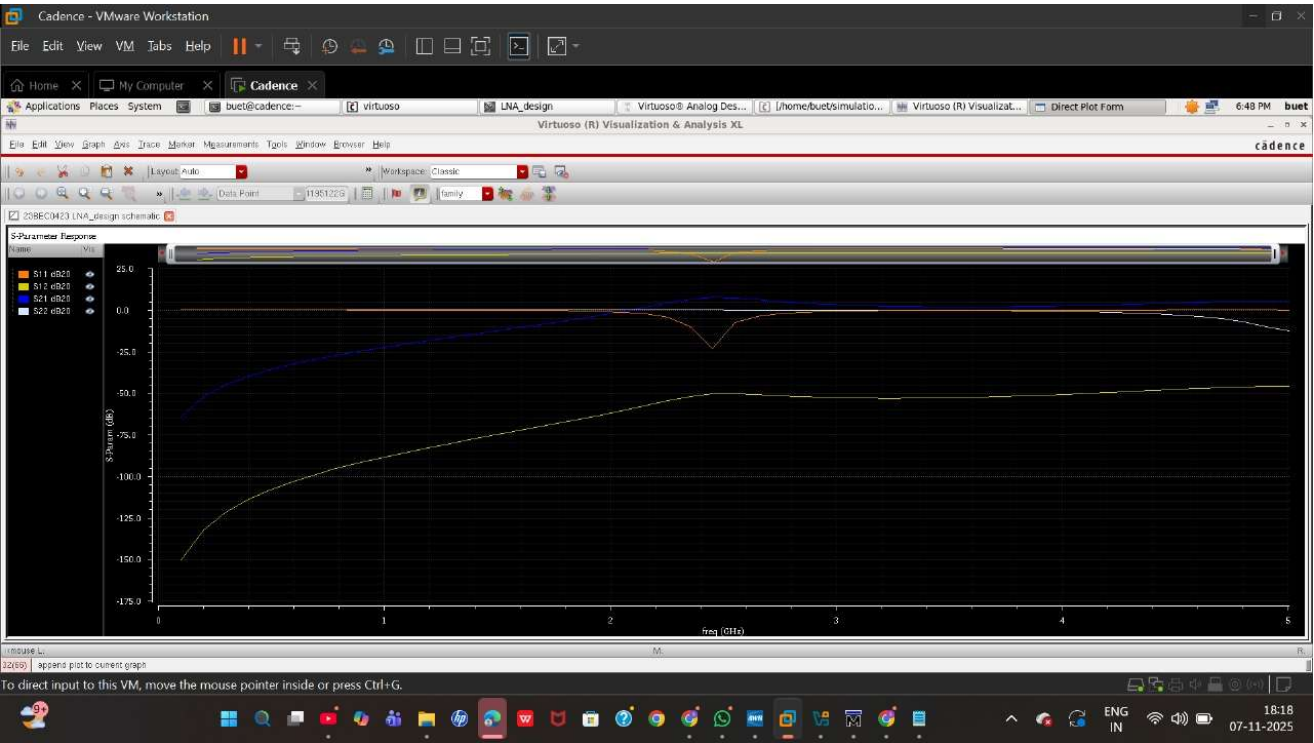
S21



S22



OVERALL S PARAMETERS



GAIN PLOT



SIMULATION RESULTS

S-Parameters

- **S11** $\approx$ **-12 to -15 dB** at 2.4 GHz $\rightarrow$ good input match
- **S21** $\approx$ **12-18 dB** across 2.4 GHz $\rightarrow$ strong gain
- **S22** $\approx$ **-10 dB** $\rightarrow$ acceptable output match

Noise Analysis

- Noise Figure (NFmin): **2-3 dB**
- Spot noise at resonance shows minimum at 2.4 GHz

Stability

- **Rollet K factor** > 1 across all frequencies $\rightarrow$ unconditionally stable
- μ factor above unity

Post-Layout Results

- Slight reduction in gain (by 1-2 dB) due to parasitics
- NF increases by ~ 0.2 dB
- Resonant frequency shifts slightly $\rightarrow$ tuned using layout adjustments

CONCLUSION:

- A CMOS Low Noise Amplifier suitable for 2.4-GHz wireless applications has been successfully designed, simulated, and validated using Cadence Virtuoso. The implemented LNA achieves low noise figure, high gain, stable operation, and good input matching. The project demonstrates complete RFIC design flow including schematic design, simulation, layout, verification, and post-layout optimization.
- The methodology can be extended to other RF front-end circuits such as mixers, VCOs, and PA drivers.

INDIVIDUAL SECTION (23BEC0467):

We have been divided into two groups

My teammate Deepak and I have done the Verilog project (part 1).

Deepak Naidu has found the resources for the project and have explained me the working of the machine.

I have executed the program and verified the result.

The corresponding waveform was attached above.